# PRACTICAL – 10

Name: K.Vaishali
Roll No: 2520030117
Sec: 8

## PART 1: INODE STRUCTURES, HARD LINKS AND SYMBOLIC LINKS

### Aim

To investigate inode structures using commands such as ls -i, stat, and find, create hard links and symbolic links, and explain their impact on inode allocation.

### Theory

An inode is a data structure in Linux that stores information about a file, such as file size, permissions, owner, timestamps, and location of data blocks. Each file has an inode number.

- A hard link points to the same inode as the original file.
- A symbolic link has its own inode and stores the path of the original file.
- Creating a hard link does not create a new inode.
- Creating a symbolic link creates a new inode for the symbolic link.

### Commands Used

| Command | Purpose |
|---|---|
| ls -i | Displays inode number |
| stat | Displays detailed file information |
| find | Searches for files |
| ln | Creates a hard link |
| ln -s | Creates a symbolic link |

Algorithm

1. Start.
2. Create a sample file.
3. Display its inode using ls -i.
4. Display file details using stat.
5. Create a hard link using ln.
4. Compare the inode numbers.
7. Create a symbolic link using ln -s.
8. Compare the inode numbers again.
U. Use find to locate the files.

10. Observe the inode allocation.
11. Stop.

## Commands / Program

```
echo "Hello Linux" > original.txt

ls -i original.txt

stat original.txt

ln  original.txt  hardlink.txt

ls -i original.txt hardlink.txt

ln -s original.txt symlink.txt

ls -i original.txt hardlink.txt symlink.txt

stat original.txt
stat hardlink.txt
stat symlink.txt

find . -name "original.txt" -o -name "hardlink.txt" -o -name "symlink.txt"
```

## Expected Observation

original.txt → Same inode
hardlink.txt → Same inode
symlink.txt → Different inode
The hard link and original file share the same inode, while the symbolic link has a separate inode.

## Flow Chart

START
↓
Create original.txt
↓
Check inode using ls -i
↓
Check details using stat
↓
Create Hard Link
↓
Compare inode numbers
↓
Create Symbolic Link
↓

Compare inode numbers
↓
Use find command
↓
Observe inode allocation
↓
STOP

### Output:

### Result
The inode structure was successfully investigated using ls -i, stat, and find. It was observed that hard links share the same inode, while symbolic links have a separate inode.

## PART 2: FILE READING AND WRITING USING mmap()

### Aim
To design a program using mmap() to perform file reading and writing and compare memory-mapped I/O with traditional read()/write() operations in terms of efficiency and implementation complexity.

### Theory
mmap() maps a file into the process's virtual memory. After mapping, the program can access the file directly through a memory pointer.
Traditional file I/O uses read() and write() system calls to transfer data between the file and a buffer.

### Traditional I/O
open() → read() → modify → write() → close()

### Memory-Mapped I/O
open() → mmap() → access memory → msync() → munmap() → close()
Memory-mapped I/O can be efficient for large files and repeated or random access because the program works directly with memory.

### Algorithm
1. Start.
2. Open the file using open().
3. Get the file size using fstat().
4. Map the file into memory using mmap().
5. Read the file contents through the mapped memory.
4. Modify the contents.
7. Synchronize changes using msync().

8. Unmap the file using munmap().
9. Close the file.
10. Stop.

## C Program

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main()
{
    int fd;
    struct stat st;
    char *data;

    fd = open("mmapfile.txt", O_RDWR);

    if (fd == -1)
    {
        perror("Error opening file");
        return 1;
    }

    if (fstat(fd, &st) == -1)
    {
        perror("Error getting file size");
        close(fd);
        return 1;
    }

    data = mmap(NULL, st.st_size,
                PROT_READ | PROT_WRITE,
                MAP_SHARED, fd, 0);

    if (data == MAP_FAILED)
    {
        perror("mmap failed");
        close(fd);
        return 1;
    }

    printf("Original content:\n%s\n", data);

    /* Write data using memory mapping */
```

```
    if (st.st_size >= 5)
    {
        memcpy(data,  "HELLO",  5);
    }

    msync(data, st.st_size, MS_SYNC);

    printf("File modified successfully.\n");

    munmap(data, st.st_size);
    close(fd);

    return 0;
}
```

## Compilation

gcc mmap_program.c -o mmap_program

## Execution

Create the input file:
echo "Hello Linux World" > mmapfile.txt
Run:
./mmap_program
Check the file:
cat mmapfile.txt

## Sample Output

Original content:
Hello Linux World

File modified successfully.
The file becomes:
HELLO Linux World

## Comparison: mmap() vs read()/write()

| Feature | read()/write() | mmap() |
|---|---|---|
| Method | Uses system calls | Maps file into memory |
| Efficiency | Good for sequential access | Efficient for repeated/ random access |
| Implementation | Simple | More complex |
| Data access | Through buffers | Directly through memory |
| System calls | More read/write calls may be required | Fewer explicit I/O calls after mapping |

| | | |
|---|---|---|
| Suitable for | Simple file operations | Large files and frequent access |

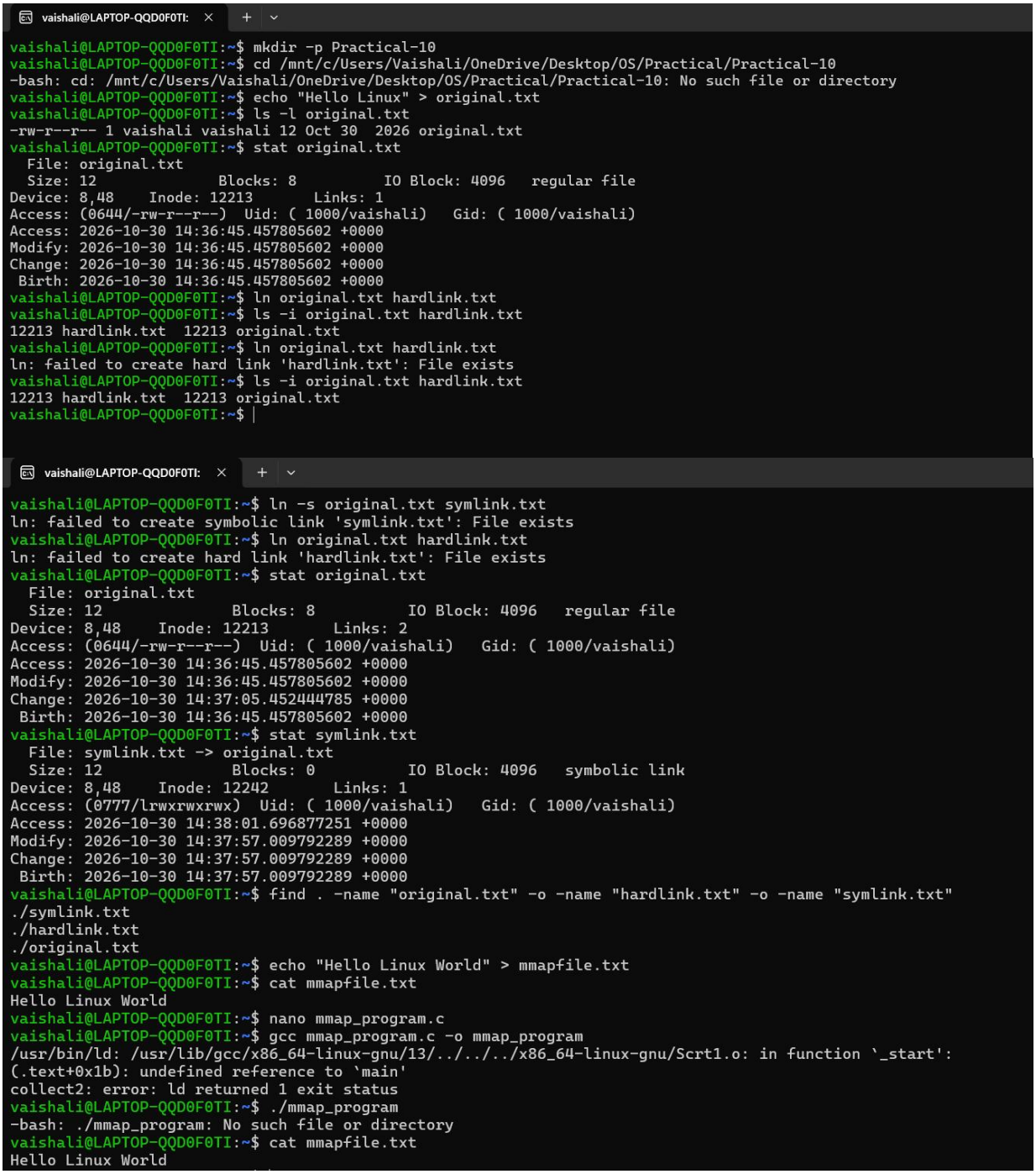
Flow Chart

START
↓
Open File
↓
Get File Size using fstat()
↓
Map File using mmap()
↓
Read File Through Memory
↓
Modify File Data
↓
msync()
↓
munmap()
↓
close()
↓
STOP

## Observation

The file was successfully read and modified using mmap(). Memory-mapped I/O provides direct memory-based access and can be efficient for large or repeatedly accessed files. However, it is slightly more complex than traditional read() and write() operations.

## Output:




## Result

The program successfully performed file reading and writing using mmap(). Memory-mapped I/O was compared with traditional read()/write() operations based on efficiency and implementation complexity.